Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Web Framework (1)

Framework

- A partial application that consists of many class-based components that specially intended to application development

- To use the libraries or features that provided by framework, developer must learn the rules of the framework, because every framework has their own rules

Framework Characteristics

- Support extensions (feature adding / improving)
- Made as simple as possible, but not simpler than without one
- Must have documentation and how-to
- Scalable dan reusable
- Skeletal or assembly structured

Framework Trade-off

- Advantages:
  - Reusability: speed up development time
  - Extensibility: flexible, reduced development complexity
  - Same coding style

- Disadvantages:
  - Learning curve (developer must learn the framework rules)
  - Possible to unknown bugs caused by library or environment dependency
  - System flow control isn't known by developer, so you don't know what is happening inside

Famous PHP Frameworks

- Laravel
- CodeIgniter
- Yii (Yes It Is)
- CakePHP
- Zend
- Symfony
- etc

Best Framework in the world (by TechnoStacks, 2021)

1. Laravel
2. CodeIgniter
3. Symfony
4. Yii 2
5. Phalcon
6. CakePHP
7. FuelPHP
8. Slim

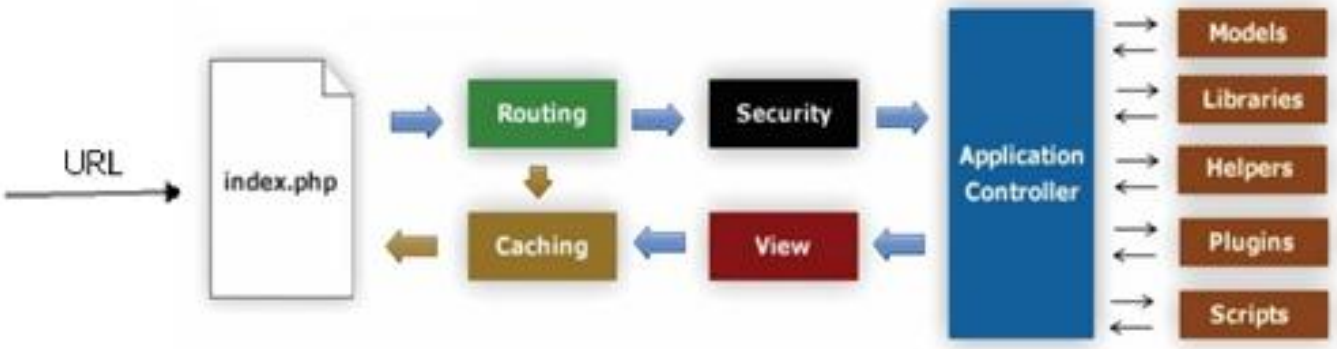
Laravel

- First rank for most used framework in US, and probably in the world
- But for Indonesia, the award goes to CodeIgniter

Why Laravel ?

- Open Source (Free)
- MVC-based
- Many extending libraries
- Complete documentation
- Large developer community
- Not-so-hard to understand and learn

MVC in Laravel



When a URL is opened, the index.php file will be read. In this file, Laravel checks if the URL is registered in the routing that has been created. If there is a mapping and the Security is fulfilled (if required, authentication for example) then Laravel will call the Controller that mapped by routing. Inside the Controller, all the back-end logic is executed, then it can return the View to the browser. This view is stored in Caching for faster processing afterwards.

Laravel Framework

Step by step basic

Installing the Laravel

1. Install via Composer (see the Practicum Book)
2. Setting the PHP environment (see the Practicum Book)
3. Open command prompt / console / terminal, go to the project ro app folder, run:

   php artisan serve

Open the browser

Laravel

Documentation

Laravel has wonderful, thorough documentation covering every aspect of the framework. Whether you are new to the framework or have previous experience with Laravel, we recommend reading all of the documentation from beginning to end.

Laracasts

Laracasts offers thousands of video tutorials on Laravel, PHP, and JavaScript development. Check them out, see for yourself, and massively level up your development skills in the process.

Laravel News

Laravel News is a community driven portal and newsletter aggregating all of the latest and most important news in the Laravel ecosystem, including new package releases and tutorials.

Vibrant Ecosystem

Laravel's robust library of first-party tools and libraries, such as Forge, Vapor, Nova, and Envoyer help you take your projects to the next level. Pair them with powerful open source libraries like Cashier, Dusk, Echo, Horizon, Sanctum, Telescope, and more.

Basic Routing

- When using Laravel, we can't access the URL with page that have .php extension, but we must access it via a route (segmented URL). Formats:

```
http://localhost:8000/[segment1]/[segment2]/...
```

- Routing settings are in the routes/web.php file. Here we can set the URL mapping with the action we want to perform. The method can also be set if it only accepts certain methods (post, get, or others).
- Routing can be set with parameters, groups, prefixes, or middlewares

Basic Routing

```
// call login.blade.php view that stores inside resources/views folder
Route::get('/login', function () {
    return view('login');
});

// if only call a view (no logic), this is a shortcut
Route::view('/', 'welcome');

// call a function from a Controller
Route::post('/auth', [SiteController::class, 'auth']);

// or like this, the 'string' version
Route::get('/product', 'App\Http\Controllers\ProductController@index');
```

Basic View

- Laravel uses Blade template-engine
- View file is stored inside resources/views folder
- View can be set directly inside Router or called from Controller function:

```
view('[file_name]',$data);
```

- [file_name] is the file name without .php.
- $data is optional. $data is set if there is data that you want to pass to the view, so you can use that data inside the view

Basic Controller

- Controller file is stored inside app/Http/Controllers folder
- You can create the Controller manually or generate it with script:

```
php artisan make:controller SiteController
```

- If Controller will be used for data management with database, then the above command can be added –resource, so the default functions that will be used for data management are generated automatically by Laravel

```
php artisan make:controller ProductController --resource
```

- For this Resource Controller, add this in Routing for all data management URL routes:

```
Route::resource('product', ProductController::class);
```

Accessing data passed by Controller to View

- Data is an array
- How to access it inside the view by access the associative (key) index:

```
//controller

$data['rebus'] = 'mie';

return view('welcome', $data);

//view welcome.blade.php

{{ $rebus }} <!-- output: mie -->
```

Using Asset Files

- Asset files (CSS, JS, or image) is stored in public folder so you can use it globally inside view by a function with a relative path URL:

```
{{ asset([file path file in public folder]) }}
```

- Common practice is you create a folder named 'assets' or so inside public folder and place the assets inside it
- So if you have a CSS file named 'style.css' inside assets folder:

```
<link rel="stylesheet" href="{{ asset('style.css') }}" />
```

- Or an image named 'apa.jpg' inside assets/img folder:

```
<img src="{{ asset('assets/img/apa.jpg') }}" />
```

Famous Node.js Frameworks

- Express.js
- NestJS
- Hapi
- AdonisJS
- etc

Simple server with Hapi

```
// load Hapi module
const Hapi = require('@hapi/hapi');

(async () => {
  // create HTTP server
  const server = Hapi.server({ host: 'localhost', port: 3000 });

  // run the HTTP server
  await server.start();
  console.log('server start at ', server.info.uri);
})()
```

Routing with Hapi

```
server.route([
  {
    method: 'GET',
    path: '/',
    handler: () => ({ message: 'You are doing GET' }),
  },
  {
    method: 'POST',
    path: '/',
    handler: () => ({ message: 'You are doing POST' }),
  },
]);
```

Any question?

Exercise

Create controller **Lat1Controller** and create index function

```
....
     public function index(){
        $data["nama"]="Agus";
        $data["asal"]="Bandung";
        return view('v_latihan1', $data);
     }
....
```

Exercise [contd.]

Create view **v_latihan1.blade.php**

```
<!DOCTYPE html>
<html>
  <head><title>Latihan 1</title></head>
  <body>
     Nama : {{$nama}} <br/>
     Asal : {{$asal}} <br/>
  </body>
</html>
```

Exercise [contd.]

Add the route in **routes/web.php**

```
....
Route::get('/lat1',
   'App\Http\Controllers\Lat1Controller@index');
....
```

Exercise [contd.]

Test it:

http://localhost:8000/lat1

Nama : Agus
Asal : Bandung

Exercise [contd.]

- Create a function in controller Lat1:

```
...
   public function method2(){
      $data['title'] = "Daftar Mahasiswa";
      $data['daf_mhs'] = array(
         array("nama" => "Agus", "asal" => "Bandung"),
          array("nama" => "Budi", "asal" => "Jakarta"),
         array("nama" => "Roni", "asal" => "Surabaya")
      );
      return view('v_latihan2', $data);
   }
...
```

Exercise [contd.]

Create view **v_latihan2.blade.php**

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{$title}}</title>
  </head>
  <body>
   <h3>{{$title}}</h3>
   <table border="1">
     <tr><th>No</th><th>Nama</th><th>Asal</th></tr>
     @php $no = 1; @endphp
     @foreach($daf_mhs as $mhs)
     <tr>
       <td>{{$no++}}</td>
       <td>{{$mhs['nama']}}</td>
       <td>{{$mhs['asal']}}</td>
     </tr>
     @endforeach
   </table>
  </body>
</html>
```

Exercise [contd.]

Add the route in **routes/web.php**

```
....
Route::get('/lat1/m2',
   'App\Http\Controllers\Lat1Controller@method2');
....
```

Exercise [contd.]

Test it:

http://localhost:8000/lat1/m2

Daftar Mahasiswa

No	Nama	Asal
1	Agus	Bandung
2	Budi	Jakarta
3	Roni	Surabaya

References

- https://laravel.com/docs/master

Fakultas Informatika
School of Computing
Telkom University

THANK YOU